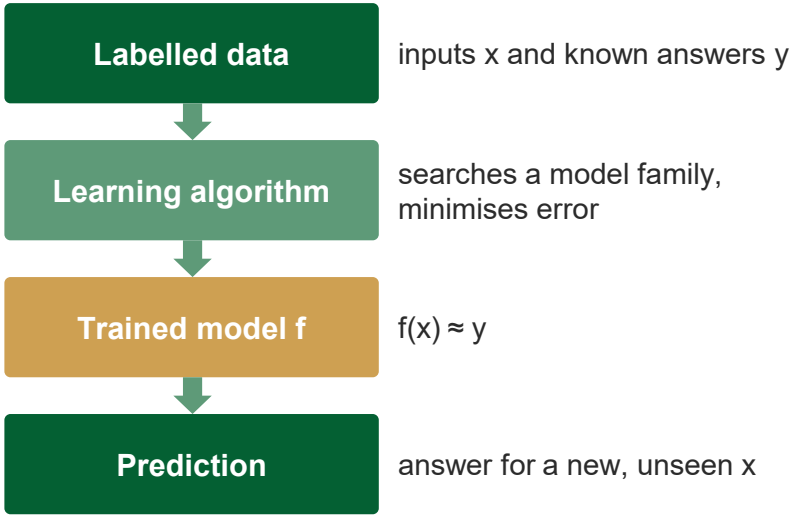
Supervised Learning: Classification

Supervised learning

Supervised learning is machine learning from examples that already have the correct answer (a label), so the model can predict the answer for new examples.



Key ideas

- **Labels are the teacher:** each example has the right answer attached.
- **Goal:** predict well on new data, not memorise the training set.
- **Labels cost money:** experts, time, and disagreements between labellers.
- **Garbage in, garbage out:** the model copies errors and biases in the labels.

Note: Split the data first, tune on validation folds, and touch the test set once.

Two types of supervised learning

Classification predicts a category (a class) for each example. Regression predicts a number on a continuous scale.

	Classification	Regression
Target	A category from a fixed set	A number on a continuous scale
Example questions	Is this email spam? Which digit is this?	What will this house sell for? How many days in hospital?
Model output	A class, usually with a probability	A number
Measured by	Accuracy, precision, recall, F1, AUC	MAE, RMSE, R^2
Covered	**Today**	Next lecture

Same data, two framings: “Will this patient stay more than 7 days?” is classification. “How many days will they stay?” is regression. Pick the one that matches the decision.

Kinds of classification problems

A class is one of the possible categories the target can take. Problems differ in how many classes there are and how many each example can have.

1 Binary

Two classes. Spam or not spam; fraud or genuine; disease or healthy.

2 Multi-class

Exactly one of several classes. Which digit 0-9? Which species of flower?

3 Multi-label

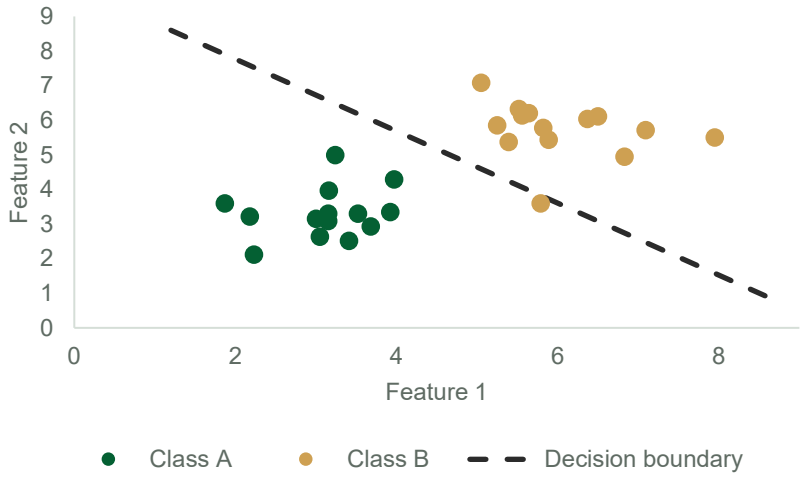
Several labels at once. A film tagged Action and Comedy; an X-ray with two findings.

Imbalanced classes: when one class is rare (fraud is often under 1% of transactions), accuracy misleads.

Everyday examples: disease diagnosis, loan default, customer churn, sentiment of a review, handwriting recognition, image tagging.

How a classifier decides

A decision boundary is the line or surface in feature space where the model's prediction switches from one class to another.

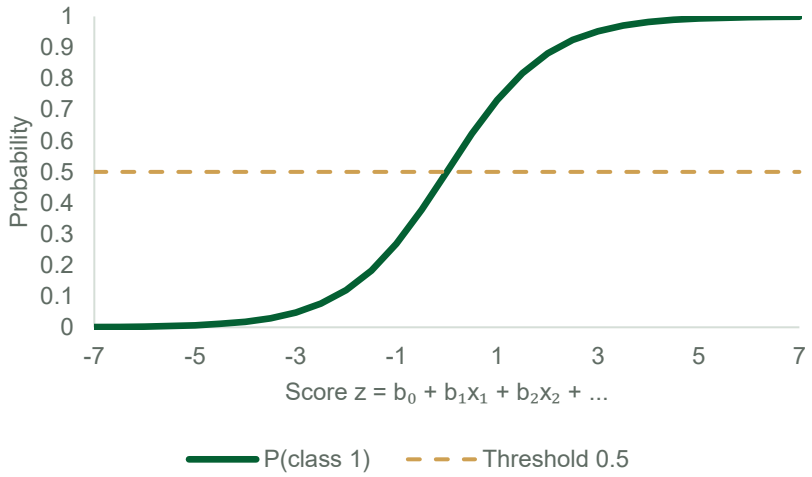


Three ideas

- **Boundary:** the line or surface that separates the class regions.
- **Probability:** most models output a score such as P(spam) = 0.83.
- **Threshold:** turns the score into a class (spam if at least 0.5).
- **Models differ** mainly in the shape of boundary they can draw.

Logistic regression

Logistic regression is a linear classifier that estimates the probability of the positive class by passing a weighted sum of the features through the S-shaped sigmoid function.



How it works

- Weighted sum of features gives a score z.
- Sigmoid turns z into a probability between 0 and 1.
- Predict class 1 if the probability is at least the threshold.
- Coefficients are readable: e^b is the odds ratio.

Key hyperparameters

- C: regularisation (small C = simpler model)
- Penalty: L2 shrinks weights, L1 drops features
- Class weights: up-weight the rare class

Use when: you need a fast baseline, probabilities or explainable coefficients.

Avoid when: the boundary is strongly curved and you have no engineered features.

k-nearest neighbours (k-NN)

k-NN is a classifier that labels a new example by a majority vote of the k training examples closest to it.



How it works

- Stores the training data; no real training step.
- Finds the k closest points to a new example.
- Majority vote (or distance-weighted vote) gives the class.
- Features must be scaled so no single feature dominates distance.

Key hyperparameters

- k: small = noisy, large = too smooth (try odd 3-15)
- Distance: Euclidean, Manhattan
- Weights: uniform or by distance

Use when: the dataset is small and the boundary is irregular.

Avoid when: data is large (slow predictions) or has many features.

Naive Bayes

Naive Bayes is a probabilistic classifier that applies Bayes' rule and assumes the features are independent of each other once the class is known.

Worked example: is an email containing “free” spam?

Quantity	Value
P(spam)	0.40
P(“free” | spam)	0.30
P(“free” | not spam)	0.02
Spam score	0.40 × 0.30 = 0.120
Not-spam score	0.60 × 0.02 = 0.012
P(spam | “free”)	**0.120 / 0.132 = 0.91**

How it works

- Bayes' rule: P(class | features) ∝ P(class) × P(each feature | class).
- Naive: treats features as independent given the class.
- Variants for numbers, word counts, or present/absent features.

Key hyperparameters

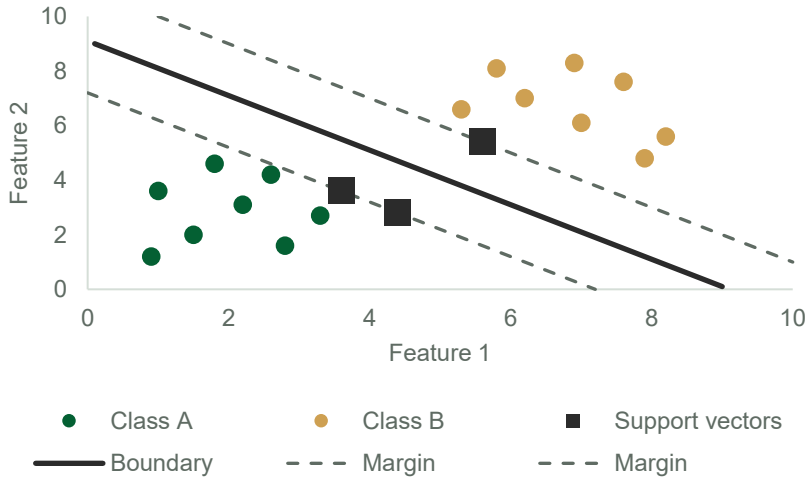
- Smoothing (alpha): avoids zero probabilities
- Variant: Gaussian, multinomial or Bernoulli

Use when: text and spam filtering, tiny datasets, or a very fast baseline.

Avoid when: you need trustworthy probabilities or features are strongly linked.

Support vector machines (SVM)

An SVM is a classifier that finds the boundary separating the classes with the widest possible margin, the gap to the nearest training points.



How it works

- Finds the boundary with the widest margin between classes.
- Only the support vectors (points on the margin) matter.
- Soft margin: C trades margin width against mistakes.
- Kernel trick allows curved boundaries.

Key hyperparameters

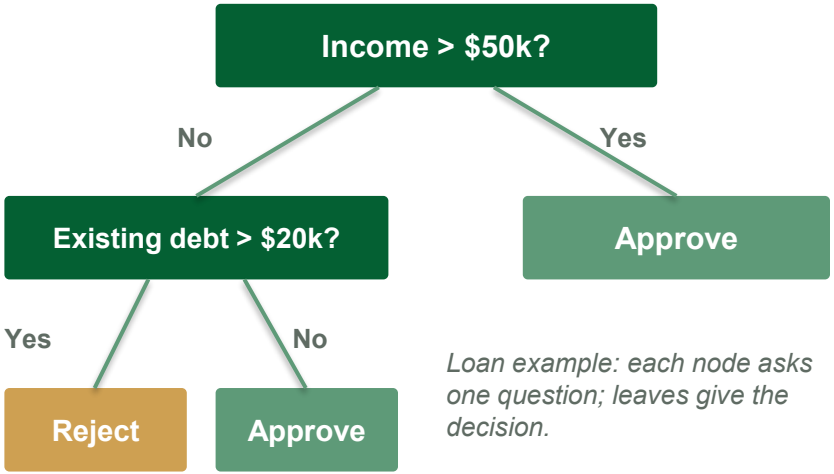
- C: high = narrow margin, risk of overfit
- Kernel: linear (text) or RBF (curved)
- Gamma (RBF): high = wiggly boundary

Use when: medium-sized data with many features, such as text.

Avoid when: data is very large, or you need probabilities and explanations.

Decision trees

A decision tree is a classifier that predicts through a sequence of yes/no questions about the features, learned from data, ending in a leaf that gives the class.



Loan example: each node asks one question; leaves give the decision.

How it works

- Asks one yes/no question at a time.
- Each split makes the groups as pure as possible (Gini or entropy).
- A leaf predicts the majority class of its rows.
- Boundary is made of axis-aligned boxes.

Key hyperparameters

- Max depth: limits how many questions in a row
- Min samples per leaf: bigger leaves, smoother model
- Criterion: Gini or entropy

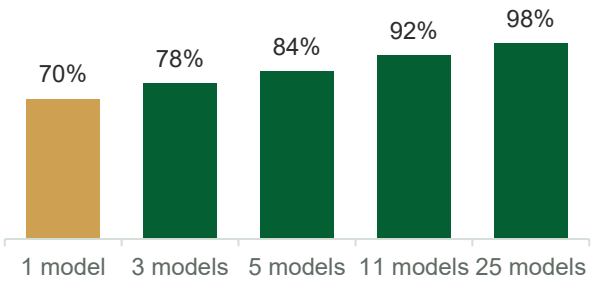
Use when: people need to read and check the rules.

Avoid when: used alone on noisy data: deep trees overfit and are unstable.

Ensemble learning

An ensemble combines the predictions of several models into one prediction that is usually more accurate and more stable than any single model.

Majority vote of models that are each 70% accurate



The vote is wrong only if most models are wrong at the same time.

1 Bagging (parallel)

Many models on random samples vote. Cuts variance. E.g. random forest.

2 Boosting (sequential)

Each new model fixes the last one's errors. Cuts bias. E.g. XGBoost.

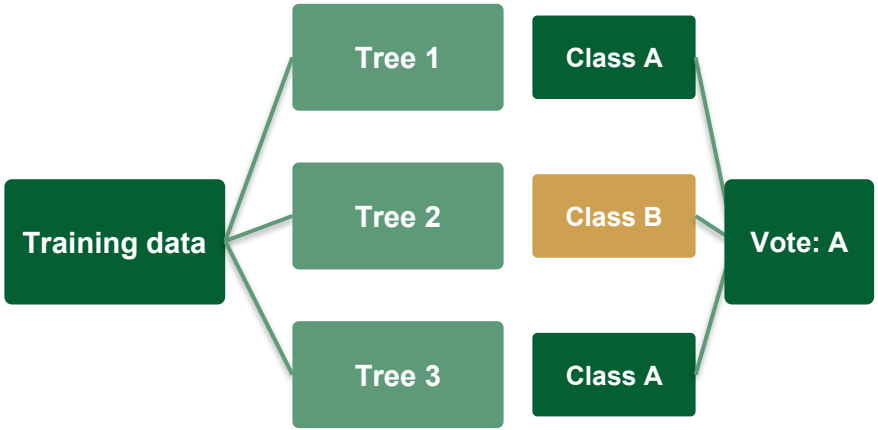
3 Voting and stacking

Combine different model types; stacking learns how to weigh them.

The catch: this only works if the models make different mistakes. Copies that err together gain nothing, so ensembles add diversity.

Bagging and random forests

Bagging (bootstrap aggregating) trains many copies of a model on random samples of the rows and combines them by voting. A random forest is bagged decision trees that also pick random features at each split.



Each tree sees a different bootstrap sample and random features.

How it works

- Draw many bootstrap samples of the rows.
- Grow a deep tree on each, using random features per split.
- Majority vote across trees gives the class.
- Out-of-bag rows give a free validation estimate.

Key hyperparameters

- Number of trees: more = stabler, slower
- Max features per split: adds diversity
- Max depth / min samples per leaf

Use when: you want a strong default on tables with little tuning.

Avoid when: you need one small, readable model, or very fast predictions.

Bagging algorithms

A bootstrap sample is a random sample of rows drawn with replacement, the same size as the training set: some rows repeat and about a third are left out.

Algorithm	What is randomised	Base model	Typical use
Bagged trees	Rows (bootstrap samples)	Deep decision tree	The original idea; teaching
Random forest	Rows + features at each split	Deep decision tree	Strong default for tables
Extra trees	Rows, features and cut-off points	Deep decision tree	Faster; even lower variance
Balanced random forest	Rows, sampled equally per class	Deep decision tree	Rare classes (fraud, disease)

Example, loan default: a random forest of 500 trees, each trained on a different bootstrap sample of past loans. For a new application, 372 trees say “default” and 128 say “repay”: the forest predicts default, with an estimated probability of 372 / 500 = 0.74.

Boosting

Boosting trains models one after another, each new model focusing on the errors of those before it, and adds them together into one strong model.

1. **Simple first guess**
2. **Measure the errors**
3. **Fit a small tree to the errors**
4. **Add it, scaled by learning rate**

How it works

- Start with a simple guess (for example the class ratio).
- Fit a small tree to the current errors.
- Add it with a small step size (learning rate).
- Repeat; stop early when validation stops improving.

Key hyperparameters

- Number of trees + early stopping
- Learning rate: lower = safer, needs more trees
- Tree depth (3-8); row and column subsampling

Use when: you want top accuracy on tabular data.

Avoid when: labels are very noisy or there is no time to tune.

Boosting algorithms

A weak learner is a model only slightly better than guessing, such as a very shallow tree. Boosting combines many weak learners into a strong one.

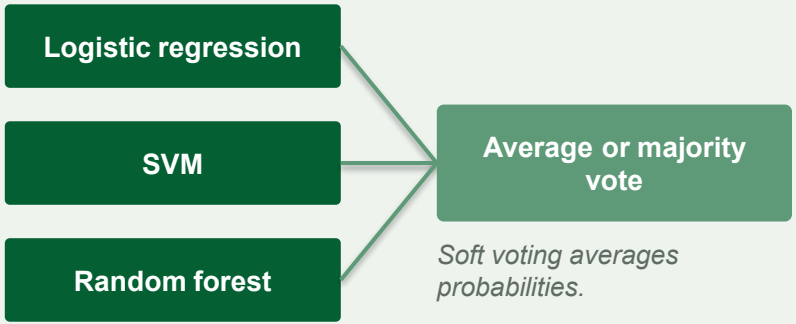
Algorithm	Year	Key idea	Best for
AdaBoost	1997	Re-weights misclassified rows; very small trees (stumps)	Simple, clean data; face detection
Gradient boosting	2001	Each tree fits the remaining errors	The general method behind the rest
XGBoost	2016	Regularised, handles missing values, parallel	Strong all-round default on tables
LightGBM	2017	Histogram bins, leaf-wise growth	Large datasets, speed
CatBoost	2018	Native categorical features, less leakage	Many categorical columns

Example, card fraud: tree 1 catches the obvious fraud (huge amount, new country). Tree 2 focuses on the frauds tree 1 missed, tree 3 on what is still wrong, and so on for a few hundred small trees, stopping when validation recall stops improving.

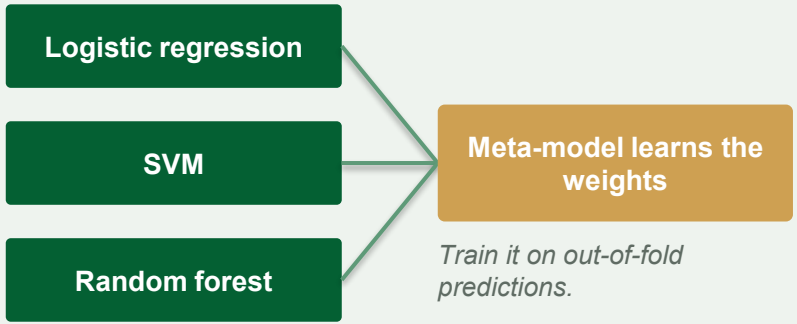
Voting and stacking

Voting combines different models by majority vote or by averaging their probabilities. Stacking trains a final model (a meta-model) to learn how best to combine them.

Voting



Stacking



Leakage warning: a stacking meta-model must learn from out-of-fold predictions, never from predictions on rows the base models were trained on.

The confusion matrix

A confusion matrix is a table that counts a classifier's predictions for each actual class against each predicted class, showing exactly which mistakes it makes.

	Predicted: Positive	Predicted: Negative
Actually sick	**TP 85** Caught it	**FN 15** Missed it
Actually healthy	**FP 10** False alarm	**TN 90** Correctly cleared

Reading it

- **TP:** predicted positive, truly positive.
- **FN:** missed a real positive (Type II error).
- **FP:** false alarm on a negative (Type I error).
- **TN:** correctly cleared a negative.
- **Multi-class:** an n × n grid; off-diagonal cells show which classes get confused.

Ask first: which mistake costs more here, a missed case (FN) or a false alarm (FP)? The answer picks the metric and the threshold.

Accuracy, precision, recall and F1

Precision is the share of predicted positives that are truly positive. Recall is the share of actual positives the model finds. F1 is the balance of the two.

Metric	Formula	Our example	The question it answers
Accuracy	(TP + TN) / total	175 / 200 = 0.875	How often is it right overall?
Precision	TP / (TP + FP)	85 / 95 = 0.895	When it says positive, is it right?
Recall	TP / (TP + FN)	85 / 100 = 0.850	Of all real positives, how many did we find?
Specificity	TN / (TN + FP)	90 / 100 = 0.900	Of all real negatives, how many did we clear?
F1 score	2PR / (P + R)	0.872	One number balancing precision and recall

R Favour recall

Missing a case is costly: cancer screening, fraud.

P Favour precision

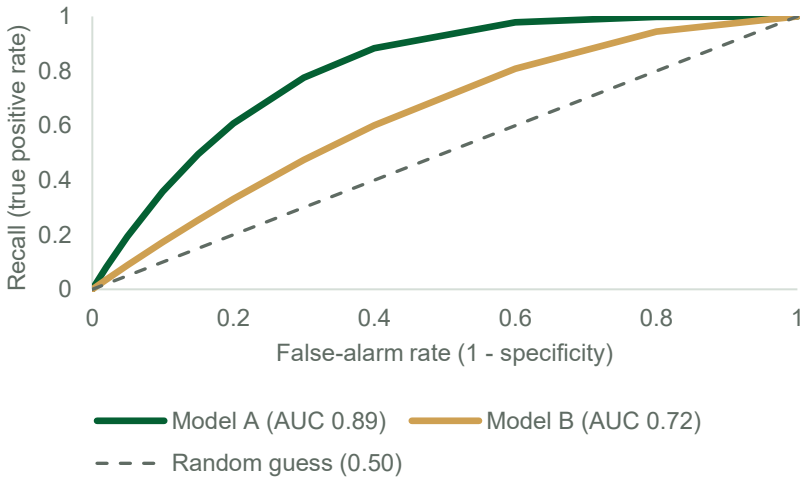
False alarms are costly: blocking real email.

F Use F1

Classes are imbalanced and both errors matter.

ROC curves and AUC

A ROC curve plots recall against the false-alarm rate for every possible threshold. AUC, the area under it, runs from 0.5 (guessing) to 1 (perfect).



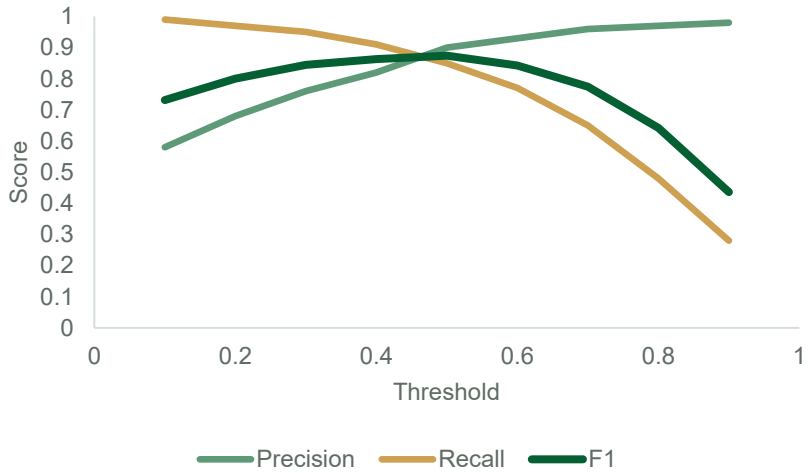
Reading it

- **ROC:** recall vs false alarms as the threshold moves.
- **AUC:** 1.0 is perfect, 0.5 is guessing. Top-left is better.
- **Lower threshold:** more positives flagged, recall up, precision down.
- **Choose it** on validation data: best F1, a recall target, or lowest cost.
- **Rare positives:** prefer the precision-recall curve.

AUC ignores the threshold: good for comparing models, but you still have to pick one. That is the next two slides.

The decision threshold

The decision threshold is the cut-off probability at or above which the model predicts the positive class.



How it works

- **Rule:** predict positive if the probability is at least the threshold.
- **Lower it:** more cases flagged; recall up, precision down.
- **Raise it:** fewer, surer cases; precision up, recall down.
- **Same model:** the threshold changes decisions, not the model or its AUC.
- **Choose it** on validation data, never on the test set.

0.5 is a default, not a law: it only suits balanced classes where both mistakes cost the same.

Same model, three thresholds

An operating point is the threshold you choose together with the precision and recall it gives. Choosing it is a decision about which mistakes cost more.

Threshold	TP	FP	FN	TN	Precision	Recall	F1	Good for
0.3	95	30	5	70	0.76	0.95	0.84	Screening: miss nobody
0.5	85	10	15	90	0.89	0.85	0.87	Balanced costs
0.7	65	3	35	97	0.96	0.65	0.77	Costly false alarms

1 By F1

Pick the threshold with the best F1 on validation data.

2 By a target

E.g. recall of at least 95%, then the best precision that meets it.

3 By cost

If a miss costs 20 false alarms, minimise 20 × FN + FP: 0.3 wins.

Re-check the threshold whenever class balance or costs change, for example at a new hospital.

Imbalanced classes

A dataset is imbalanced when one class is much rarer than the others, such as fraud in under 1% of card transactions.

1 Right metric

Recall, precision, F1, PR-AUC or balanced accuracy, not accuracy.

2 Stratified splits

Keep the rare class in every fold and in the test set.

3 Class weights

Make mistakes on the rare class cost more during training.

4 Tune the threshold

Lower it to catch more rare cases; check precision.

5 Resample carefully

Over- or undersample inside training folds only, never before splitting.

6 More rare examples

Often the best fix of all, if you can get them.

The accuracy paradox: if 0.5% of transactions are fraud, always predicting “not fraud” scores 99.5% accuracy and catches nothing.